# Software-as-a-Graph: A Model-Driven Framework for Graph-Based Architectural Modeling, Diagnostic Analysis, and Counterfactual Refactoring of Publish–Subscribe Systems

Ibrahim Onuralp Yigit*

Feza Buzluca†

[DEPARTMENT], [INSTITUTION]
[STREET ADDRESS], [CITY], [POSTAL CODE], [COUNTRY]

November 13, 2026

**Abstract**

Distributed publish–subscribe architectures—such as the Robot Operating System (ROS 2), Data Distribution Service (DDS), and MQTT—form the backbone of modern robotics, cyber-physical systems, and event-driven microservices. While asynchronous topic-based communication provides spatial and temporal decoupling, it obscures topological dependencies and failure propagation pathways, creating an **Architecture-Implementation Gap** where pristine component codebases can harbor severe system-level vulnerabilities. In this tool paper, we present **Software-as-a-Graph (SaG)**, an extensible, open-source model-driven engineering (MDE) framework for graph-based architectural modeling, diagnostic quality analysis, and counterfactually verified refactoring of publish–subscribe software systems.

SaG extracts typed, multi-layer multigraph models from Architecture-as-Code descriptors, folds source-level static code metrics into a **Code Quality Penalty (CQP)**, and computes an explainable, deterministic quality attribution model grounded in ISO/IEC 25010 and ISO/IEC 25019 standards across Reliability (Fault Tolerance and Availability) and Maintainability. Over these architectural models, SaG formalizes and evaluates a catalog of nineteen publish–subscribe anti-patterns using adaptive box-plot thresholds. To remediate diagnosed technical debt, the tool features a prescriptive engine that compiles findings into typed graph-mutation refactoring operators—logical topic splitting, physical host anti-affinity reallocation, and transport QoS contract hardening. Crucially, SaG enforces a closed-loop **"verify-before-transform"** discipline: candidate refactorings are counterfactually evaluated in an in-memory simulation sandbox, admitting only edits whose cascade-impact reduction exceeds stochastic seed noise across propagation thresholds ($\overline{\Delta I} > \kappa\sigma_{\text{seed}}$).

SaG is engineered as a decoupled hexagonal architecture featuring a dual-repository pattern: a graph database adapter (`Neo4jRepository`) for persistent querying and interactive exploration, and a high-performance in-memory adapter (`MemoryRepository`) that executes over 14,000 counterfactual simulation sweeps in under 3 hours. The tool provides multi-modal interfaces serving diverse stakeholder personas, including an interactive web dashboard and graph explorer, a Python SDK, a modular CLI suite, a REST API, and automated CI/CD regression gates. We evaluate SaG across eight multi-domain benchmark scenarios (29–520 vertices) and three transcribed real-world open-source systems (Autoware.Universe, Apollo Cyber RT, and an MQTT cloud mesh). We candidly report key empirical findings and calibration realities, including ranking saturation among structural predictors (NDCG@10 $\approx$ 0.82–0.86, demonstrating that domain-grounded attribution is the primary value over scalar centrality), catalog discrimination improvements on real-world topologies ($\kappa = +0.296$ vs. chance on synthetic graphs), and a 71.0% refactoring admission rate. Finally, we articulate five

*Corresponding author. E-mail: `[CORRESPONDING-AUTHOR-EMAIL]`. ORCID: `[0000-0000-0000-0000]`.
†ORCID: `[0000-0000-0000-0000]`.

transferable lessons learnt spanning metamodel substrate adequacy, in-memory sandboxing, and simulation-in-the-loop verification to benefit the broader systems modeling community.

**Keywords:** Model-driven engineering; Software and systems modeling; Architectural models; Graph metamodeling; Anti-pattern detection; Counterfactual refactoring; Publish–subscribe middleware; Simulation-based verification; Tool paper.

# 1 Introduction

## 1.1 Context and Domain Motivation: Modeling Publish–Subscribe Systems

Distributed publish–subscribe (pub–sub) middleware frameworks—including the Robot Operating System (ROS 2), the Object Management Group's Data Distribution Service (DDS), Apache Kafka, and MQTT—form the foundational communication substrate of modern robotics, autonomous vehicles, industrial IoT, and distributed microservices (Lee et al., 2025b,a). By decoupling message producers and consumers across space (anonymous communication), time (asynchronous message delivery), and synchronization (non-blocking publish and subscribe operations) via intermediary topics and brokers, these architectures enable high scalability, modularity, and operational flexibility (Wang et al., 2015; Pallickara et al., 2007).

However, this decoupling introduces a fundamental challenge for Model-Driven Engineering (MDE) and software architecture analysis: *it obscures the end-to-end topological dependencies of the system.* In publish–subscribe environments, components do not maintain explicit point-to-point references to one another; instead, dependencies are mediated dynamically through topic contracts, message brokers, and Quality of Service (QoS) transport profiles (e.g., reliability deadlines, history depths, liveliness leases). Consequently, cascading failures, buffer overflows, broker bottlenecks, and timing jitter propagate along indirect architectural pathways that remain invisible when analyzing individual source code files or static component interfaces in isolation (Yigit and Buzluca, 2025; Basiri et al., 2016).

## 1.2 Open Challenges in Architectural Modeling and Tool Support

Despite the proliferation of model-based tools for software design and dependability analysis, engineering teams managing publish–subscribe systems face three critical tool and methodological gaps:

1. **The Architecture-Implementation Gap in MDE:** Classical static analysis platforms (e.g., SonarQube, SpotBugs) operate at the file, class, or method level, focusing on syntactic code smells and local complexity. Conversely, conventional Architecture Description Languages (ADLs) and MDE frameworks (e.g., AADL, Palladio Component Model) have historically been optimized for synchronous Remote Procedure Call (RPC) or procedural client-server paradigms (Becker et al., 2009; Feiler et al., 2006). As a result, an event-driven system can achieve pristine unit-level code quality while harboring severe architectural vulnerabilities—such as unmitigated Single Points of Failure (SPOFs), co-located high-criticality components on shared hardware nodes, or mismatched transport reliability contracts. Bridging this gap requires tools capable of lifting code-level artifacts and deployment configurations into unified, multi-layer architectural graph models.

2. **Lack of Domain-Specific Anti-Pattern Catalogs:** In object-oriented programming, mature catalogs of design anti-patterns (e.g., God Class, Shotgun Surgery) provide software modelers with standardized vocabularies and detection rules (Fowler, 1999; Brown et al., 1998). Similarly, service-oriented and microservices research has cataloged structural smells such as cyclic dependencies and shared databases (Taibi et al., 2020). However, *publish–subscribe systems lack a formalized catalog of architectural anti-patterns stated*

*over their native abstractions*—specifically topics, broker partitions, and QoS contracts. Without formal metamodel signatures and detection criteria, architectural degradation is identified reactively during catastrophic production incidents rather than proactively during design and modeling.

3. **Open-Loop Model Refactoring vs. Verified Transformation:** While search-based model engineering (SBSE) and automated refactoring tools can propose structural modifications (Harman et al., 2012; Aleti et al., 2013), they almost universally operate *open-loop*: they apply syntactic transformation rules or optimize abstract heuristic fitness functions without verifying whether the mutated architecture actually prevents failure propagation under dynamic message workloads. In safety-critical domains, unverified architectural transformations risk introducing subtle topological regressions or exacerbating cascading failure risks.

## 1.3 The SaG Framework: An MDE Tool for Publish–Subscribe Systems

To resolve these challenges, we introduce **Software-as-a-Graph (SaG)**, an open-source, extensible Model-Driven Engineering framework designed to model, diagnose, and counterfactually refactor distributed publish–subscribe architectures. SaG extracts typed, multi-layer multi-graph models from Architecture-as-Code descriptors, evaluates architectural health against international quality standards, and executes verified graph-mutation refactorings.

As shown in Figure 1, SaG integrates two coordinated modeling pathways within a unified, hexagonal architecture:

- **The Diagnostic Modeling Pathway:** Ingests declarative architecture manifests and source-level static code metrics, synthesizing them into a **Code Quality Penalty (CQP)**. It constructs a multi-layer heterogeneous dependency graph ($G_{\text{analysis}}$) spanning applications, libraries, topics, message brokers, and physical hosting nodes across four architectural layers. Grounded in **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) (ISO/IEC, 2023a,b), the tool computes a hierarchical **Reliability–Maintainability (RM)** attribution model. Reliability decomposes into **Fault Tolerance** ($FT$, cascading error reach) and **Availability** ($A$, single-point-of-failure exposure), while **Maintainability** ($M$) quantifies coupling complexity, change ripple risk, and code-level debt. Over these models, SaG executes an extensible engine detecting nineteen formalized publish–subscribe anti-patterns using adaptive box-plot thresholds, mapping findings directly to stakeholder roles (Reliability Engineers, DevOps/SREs, and Software Architects).

- **The Counterfactual Prescriptive Pathway:** Rather than proposing speculative or unverified modifications, SaG compiles diagnosed architectural pathologies into three formal graph-mutation operators: logical topic splitting, physical host anti-affinity reallocation, and transport QoS contract hardening. The tool implements a closed-loop **"verify-before-transform"** discipline: every candidate transformation is counterfactually applied in an isolated in-memory graph sandbox and evaluated against a discrete-event cascade simulator across multiple random seeds and propagation thresholds. A refactoring is admitted if and only if its measured cascade-impact reduction clears simulator seed noise at every threshold ($\overline{\Delta I} > \kappa\sigma_{\text{seed}}$), ensuring that only empirically beneficial architectural transformations are recommended.

## 1.4 Target Audience and Tool Use Cases

SaG is designed to serve four primary categories of users in the systems and software modeling community:

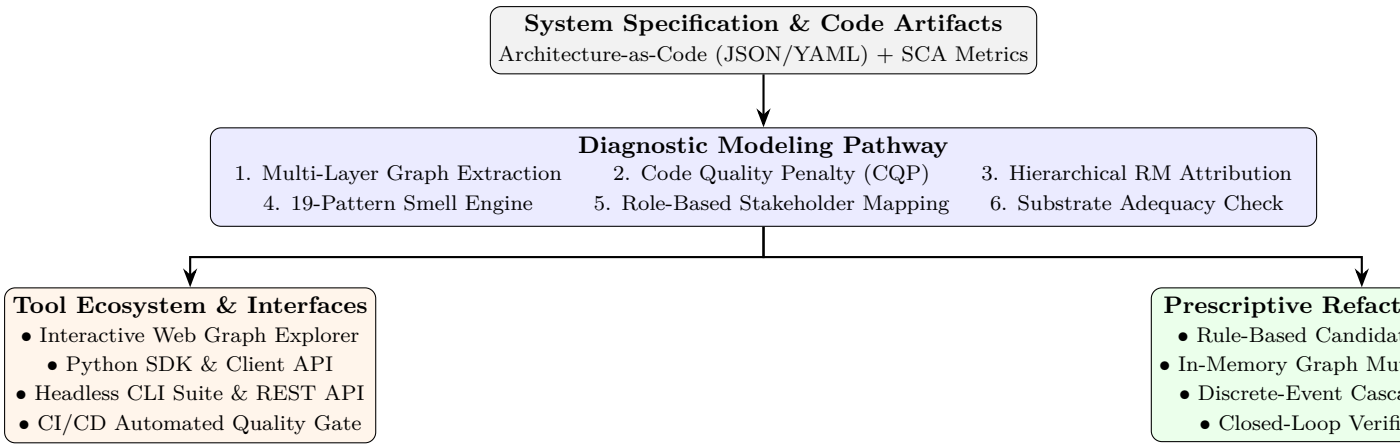



Figure 1: System architecture of the Software-as-a-Graph (SaG) framework: the multi-layer graph metamodel and diagnostic pathway serve as the shared foundation powering both multi-modal tool interfaces (Web, SDK, CLI, CI/CD) and the closed-loop counterfactual refactoring engine.

1. **Software and Systems Architects:** Architects use SaG to model event-driven topologies before deployment, visualize cross-layer dependency structures, detect architectural anti-patterns, and explore verified refactoring blueprints to eliminate structural bottlenecks.

2. **Reliability and SRE Engineers:** Site reliability engineers leverage SaG's deterministic ISO 25010/25019 quality decompositions to identify Single Points of Failure and estimate blast-radius vulnerability without performing destructive fault-injection tests on production clusters.

3. **MDE and Systems Modeling Researchers:** Researchers can utilize SaG as an extensible, open-source experimental platform for evaluating novel graph metrics, domain-specific anti-patterns, and model transformation algorithms on realistic distributed benchmarks.

4. **DevOps and Quality Assurance Teams:** DevOps practitioners integrate SaG's headless CLI and regression gating protocols into continuous integration pipelines, preventing structural technical debt regressions before code merges.

### 1.5 Contributions to the Modeling Community

In accordance with the criteria of the SoSyM Tool Paper Call, this article presents the following contributions:

1. **A Domain-Specific Graph Metamodel for Publish–Subscribe Systems (Section 3):** We formalize a typed, multi-layer multigraph metamodel $(V, E, \tau_V, \tau_E, w_V, w_E)$ capturing applications, libraries, topics, brokers, and compute hosts across four layer projections, governed by seven formal dependency derivation rules and a continuous Code Quality Penalty (CQP).

2. **A Formal Catalog of 19 Publish–Subscribe Anti-Patterns (Section 4):** We formalize nineteen architectural anti-patterns stated over pub–sub constructs across Reliability, Availability, Maintainability, and Architecture, complete with metric signatures, adaptive box-plot detection thresholds, and finding scope classifications.

3. **Tool Architecture and Implementation Techniques (Section 5):** We detail SaG's hexagonal ports-and-adapters architecture, highlighting its dual-repository design pattern

that pairs a persistent graph database (`Neo4jRepository`) with a high-throughput, in-memory sandboxed repository (`MemoryRepository`) enabling rapid counterfactual model transformations.

4. **A Closed-Loop Counterfactual Refactoring Verifier (Section 5):** We introduce a rule-based refactoring compiler coupled with three graph-mutation operators, implementing a two-level simulation verification discipline that rejects mutations failing to clear stochastic noise floor margins.

5. **Tool Usability and Multi-Modal Interfaces (Section 6):** We describe the multi-modal interaction modalities engineered into SaG, including an interactive web-based graph explorer, a Python SDK, a modular CLI, a REST API, and CI/CD quality gating with standardized exit-code protocols.

6. **Comprehensive Empirical Evaluation in Realistic Scenarios (Section 7):** We evaluate SaG across eight multi-domain benchmark scenarios (29 to 520 vertices) and three transcribed real-world open-source systems (Autoware.Universe, Apollo Cyber RT, and Cloud IoT Mesh), reporting ranking quality saturation (NDCG@10 $\approx$ 0.82–0.86), catalog discrimination characteristics ($\kappa = +0.296$ on real-world topologies), and refactoring survival rates (71.0%).

7. **Community Importance and Lessons Learnt (Section 8):** In fulfillment of SoSyM's tool paper mandate, we explicitly articulate the framework's broader significance for systems modeling and detail five transferable lessons learnt across designing, building, testing, evaluating, and disseminating the tool.

8. **Open-Source Availability and Reproducibility (Section 9):** The tool is released under the Apache 2.0 license, packaged for pip installation and Docker Compose deployment, and backed by a complete Zenodo-archived reproduction package with a persistent DOI.

### 1.6 Relationship to Prior Work

This paper builds upon and expands two prior research efforts by the authors:

1. An initial conference workshop paper (Yigit and Buzluca, 2025) introduced an early prototype of the multi-layer graph representation and evaluated a preliminary heuristic criticality score ($CS$) against a reachability loss metric on a single, small 10-application system ($\rho = 0.94$). In Section 7, we re-measure this claim across eight heterogeneous scenarios under an independent discrete-event cascade simulator, explaining why ranking correlation on directed dependency graphs saturates around $\rho \approx 0.485$ and why deterministic, role-attributed quality modeling is the true value over scalar ranking.

2. A companion research manuscript focuses specifically on a learned Graph Neural Network (GNN) predictor for blast-radius forecasting. The present tool paper is entirely distinct and self-contained: it presents the comprehensive **Software-as-a-Graph framework**, its domain metamodel, the formal 19-pattern catalog, the dual-repository architecture, the closed-loop counterfactual refactoring engine, the multi-modal tool interfaces, the realistic scenario evaluation, and the community lessons learnt.

### 1.7 Organization

The remainder of this paper is structured as follows. Section 2 reviews related work in MDE, architecture modeling, and smell detection. Section 3 defines the graph metamodel and diagnostic quality attribution framework. Section 4 details the 19-pattern anti-pattern catalog. Section 5 presents the tool architecture, implementation techniques, and counterfactual refactoring engine.

Section 6 describes tool usability, interfaces, and CI/CD integration. Section 7 evaluates the tool on benchmark scenarios and real-world systems. Section 8 articulates the tool's community importance and five key lessons learnt. Section 9 addresses threats to validity and tool availability, and Section 10 concludes with future work.

# 2 Background and Related Modeling Tools

## 2.1 Model-Driven Engineering and Architectural Description Languages

Model-Driven Engineering (MDE) elevates high-level system models to primary engineering artifacts, enabling automated analysis, formal verification, and code synthesis (Schmidt, 2006; Combemale et al., 2016). Within software and systems modeling, Architecture Description Languages (ADLs) such as the Architecture Analysis & Design Language (AADL) (Feiler et al., 2006) and the Palladio Component Model (PCM) (Becker et al., 2009) formalize structural hierarchies, component interfaces, and execution platforms for dependability and performance forecasting.

However, existing ADLs and architectural modeling tools are primarily tailored for synchronous remote procedure call (RPC), client–server interactions, or discrete hardware bus execution. In event-driven publish–subscribe (pub–sub) systems, communication is mediated through anonymous topics, message brokers, and Quality of Service (QoS) transport policies. Classical component-connector models fail to capture the subtle dynamic coupling introduced when dozens of subscribers bind to a single high-fanout topic or when broker memory partitions become exhausted under unthrottled producer bursts. SaG advances MDE by formalizing a domain-specific, typed multigraph metamodel specifically engineered to capture publish–subscribe topology semantics, multi-layer projections, and transport QoS contracts.

## 2.2 Architectural Smell and Anti-Pattern Detection Tools

In object-oriented software engineering, Fowler's refactoring catalog (Fowler, 1999) and Brown et al.'s AntiPatterns taxonomy (Brown et al., 1998) formalized recurring design flaws and remediation strategies. Suryanarayana et al. systematized design smells using structural metrics (Suryanarayana et al., 2014). In distributed microservices, Taibi et al. established empirical taxonomies of architectural smells, such as Cyclic Dependency, Shared Database, and Distributed Monolith (Taibi et al., 2020).

At the architectural level, automated smell detection is an established discipline. Mo et al. formalized architectural hotspots and evaluated their maintenance costs (Mo et al., 2015). Specialized tools such as Arcan (Fontana et al., 2017), Designite (Sharma et al., 2016), and DV8 (Cai and Kazman, 2019) detect dependency smells from reverse-engineered source-code graphs, and recent surveys document their agreements and divergence (Azadi et al., 2019; Mumtaz et al., 2021). Common patterns such as cyclic dependencies, hub-like dependencies, and unstable interfaces align conceptually with `CYCLE`, `HUB_AND_SPOKE`, and `UNSTABLE_INTERFACE` in our catalog.

However, existing tools operate on module, package, or class dependency graphs recovered from source code; *they possess no representation for topics, message brokers, partition queues, or QoS transport policies.* Consequently, they are blind to the failure modes unique to publish–subscribe middleware. Furthermore, empirical studies by Lenarduzzi et al. (Lenarduzzi et al., 2020) indicate that static rule-based tools (e.g., SonarQube) often exhibit weak correlation with real-world fault proneness. SaG addresses this by grounding its 19-pattern catalog in a multi-layer graph metamodel and evaluating detection against dynamic cascade simulation.

Software performance engineering has also cataloged performance antipatterns (Smith and Williams, 2000, 2002), which are detected in Java applications (Trubiani et al., 2023) or modeled via queueing networks (Pinciroli et al., 2023; Avritzer et al., 2025). While patterns such as the

Blob and One-Lane Bridge resemble SaG's `GOD_COMPONENT` and `BROKER_OVERLOAD`, they target execution latency, whereas SaG targets structural failure cascades. Similarly, scalability fault detectors (e.g., ScaleCheck (Stuardo et al., 2019), PCatch (Li et al., 2018), and recent distributed fault studies (Zhu et al., 2026)) target code-level thread and socket bottlenecks, whereas SaG targets pre-deployment architectural models.

### 2.3 Model Refactoring: Open-Loop Search vs. Counterfactual Verification

Automated model refactoring and architecture optimization have been approached through multiple modeling paradigms:

1. *Rule-Based Model Transformations:* Graph transformation tools apply declarative graph rewrite rules to restructure models based on predefined architectural patterns (Mens and Van Gorp, 2006).

2. *Search-Based Software Engineering (SBSE):* Multi-objective evolutionary algorithms (e.g., NSGA-II) explore architectural design spaces to optimize coupling, cohesion, or hardware cost (Harman et al., 2012; Aleti et al., 2013; Deb et al., 2002).

3. *Simulation-in-the-Loop Optimization:* The Palladio/PerOpteryx line of work (Koziolek et al., 2011) evaluates candidate architectural variants within an optimization loop using simulation and analytical models.

Despite these advances, the vast majority of architectural refactoring engines operate *open-loop* with respect to failure propagation: transformations are selected either by heuristic structural fitness functions or by unverified rule triggers. SaG introduces a closed-loop **"verify-before-transform"** discipline: candidate graph mutations are executed in an in-memory sandbox and evaluated against a discrete-event cascade simulator across multiple seeds and failure thresholds. Edits are admitted only if the measured impact reduction clears stochastic seed noise ($\overline{\Delta I} > \kappa\sigma_{\text{seed}}$), preventing regressions.

### 2.4 Explainable Quality Models and Graph Centrality

Software quality models, standardized in ISO/IEC 25010 (Product Quality) and ISO/IEC 25019 (Quality-in-Use) (ISO/IEC, 2023a,b), structure software quality into hierarchical characteristics. In distributed systems, however, predictive analytics often rely on opaque machine learning models or collapse multiple metrics into unweighted sums (Menzies and Rahman, 2018).

SaG strictly distinguishes *post-hoc attribution* (e.g., SHAP or GNN attention weights, which approximate an opaque model's decisions) from *intrinsic score decomposition*. SaG defines its Reliability–Maintainability (RM) quality model as a deterministic, weighted decomposition over standardized sub-characteristics (Fault Tolerance, Availability, Maintainability). This ensures that every computed score has auditable provenance directly mapped to stakeholder engineering roles.

Finally, while complex network theory applies graph centrality indices (betweenness (Freeman, 1977), degree, PageRank, articulation points) to microservices and call graphs (Bakhtin et al., 2025; Falci et al., 2018), our empirical evaluation reveals that multi-dimensional metrics saturate at comparable ranking accuracy with degree centrality (Section 7). SaG's principal value is therefore its deterministic, domain-grounded attribution and verified refactoring, rather than superior scalar ranking.

### 2.5 Advance Over State of the Art

Table 1 summarizes SaG's key contributions in comparison to state-of-the-art modeling and architecture analysis tools.

Table 1: Comparison of SaG with State-of-the-Art Modeling and Architecture Tools.

| **Capability / Characteristic** | **Palladio / PerOpteryx** | **AADL** | **Arcan / Designite** | **SonarQu** |
|---|---|---|---|---|
| Pub–Sub Semantics (Topics, Brokers, QoS) | No | Partial | No | No |
| Multi-Layer Heterogeneous Metamodel | Component/Hardware | Component/Bus | Class/Package | File/Lin |
| Standardized Quality Model | Custom metrics | Custom annexes | Smells only | Technical D |
| Domain-Specific Anti-Pattern Catalog | Performance only | Fault trees | OO / Microservices | Code rul |
| Code-to-Architecture Bridge (CQP) | No | No | No | Native (AS |
| Refactoring Verification Discipline | Open-loop / Sim | Analytic / Open | Open-loop | Advisory o |
| In-Memory Sandboxed Transformation | No | No | No | No |
| Multi-Modal Interfaces (Web, SDK, CLI) | Eclipse IDE | OSATE / Eclipse | CLI / Desktop | Web Dashb |
| CI/CD Regression Quality Gating | No | No | Partial | Yes |
| Open Source License | EPL | Open source | Proprietary/Academic | LGPL / Free |

# 3 Graph Metamodel and Multi-Layer System Representation

## 3.1 Heterogeneous Graph Metamodel

In the SaG framework, a distributed publish–subscribe software architecture is formalized as a typed, attributed, directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_E, w_V) \tag{1}$$

where $V$ is the set of architectural vertices, $E \subseteq V \times V$ is the multiset of directed edges, $\tau_V : V \to T_V$ partitions vertices into five distinct semantic classes, $\tau_E : E \to T_E$ assigns each edge to an architectural relationship type, $w_E : E \to \mathbb{R}^+$ maps edges to QoS-derived communication weights, and $w_V : V \to [0, 1]$ assigns normalized component weights.

The vertex type universe $T_V$ captures the fundamental building blocks of publish–subscribe architectures:

$$T_V = \{\text{Application}, \text{Library}, \text{Topic}, \text{Broker}, \text{Node}\} \tag{2}$$

- **Application ($V_{\text{app}}$):** Autonomous execution processes (e.g., ROS 2 nodes, microservice daemons) that actively produce or consume messages.
- **Library ($V_{\text{lib}}$):** Passive shared code modules or algorithmic libraries dynamically or statically linked by applications.
- **Topic ($V_{\text{topic}}$):** Named publish–subscribe communication channels routing asynchronous message streams.
- **Broker ($V_{\text{broker}}$):** Message routing and queuing intermediaries (e.g., MQTT brokers, Kafka partition leaders, DDS domain participants).
- **Node ($V_{\text{node}}$):** Physical or virtual compute hosts, containers, or virtual machines executing processes and brokers.

The edge type universe $T_E$ formalizes the operational and structural relationships among these entities:

$$\begin{aligned} T_E = \{&\text{PUBLISHES_TO}, \text{SUBSCRIBES_TO}, \text{ROUTES}, \\ &\text{RUNS_ON}, \text{CONNECTS_TO}, \text{USES}\} \end{aligned} \tag{3}$$

## 3.2 Derived Architectural Dependencies: The `DEPENDS_ON` Projection

Because pub–sub systems achieve spatial and temporal decoupling via topic mediation, direct runtime invocation dependencies are not explicitly declared in source manifests. To expose indirect failure propagation pathways, SaG defines formal derivation rules that project $G$ into

a directed analysis graph $G_{\text{analysis}} = (V, E_{\text{depends}})$, where directed edges $(u \to v)$ denote that entity $u$ functionally depends on entity $v$ (i.e., disruption of $v$ directly impacts $u$):

- **Application-to-Application:** Formed when application $A_1$ subscribes to a topic $T$ published by application $A_2$: $(A_1 \to A_2) \in E_{\text{depends}}$.
- **Application-to-Broker:** Formed when application $A$ publishes or subscribes to a topic routed through broker $B$: $(A \to B) \in E_{\text{depends}}$.
- **Application-to-Library:** Formed when application $A$ links shared library $L$: $(A \to L) \in E_{\text{depends}}$, capturing shared-dependency blast radius if $L$ suffers a defect.
- **Broker-to-Broker / Node Colocation:** Captures shared-fate host vulnerabilities when multiple critical applications or brokers are co-located on the same physical host $N$: $(A_i \to N) \in E_{\text{depends}}$.

The resulting analysis graph is stratified across four architectural layer projections:

1. **Application Layer (app):** Subgraph induced by $V_{\text{app}} \cup V_{\text{lib}}$, modeling end-to-end business logic and functional data flows.
2. **Infrastructure Layer (infra):** Subgraph induced by $V_{\text{node}}$, modeling physical and virtual compute network topology.
3. **Middleware Layer (mw):** Subgraph induced by $V_{\text{app}} \cup V_{\text{broker}}$, modeling broker mediation, partition bottlenecks, and transport queues.
4. **System Layer (system):** The complete multi-layer graph across all five vertex types, capturing cross-layer failure cascades.

### 3.3 The Code Quality Penalty (CQP): Bridging SCA and Architecture

To bridge source-level implementation health with macro-architectural risk, SaG ingests four file-level static code analysis (SCA) metrics for Application and Library components:

- Total Lines of Code (`loc`, sizing component scale),
- Average Weighted Methods per Class (`cyclomatic_complexity`, proxy for cognitive complexity),
- Lack of Cohesion in Methods (`lcom`, proxy for modularity decay),
- Code-level instability, $\text{instability_code}(v) = C_e(v)/(C_a(v)+C_e(v))$, computed from efferent and afferent coupling.

Signals are min-max normalized independently within each component type population (Applications and Libraries form distinct populations with different scale baselines). Populations with zero variance collapse to 0 rather than 1, ensuring that absent data is not penalized as worst-case debt. These normalized metrics synthesize into the continuous **Code Quality Penalty (CQP)**:

$$\begin{aligned} \text{CQP}(v) = {} & 0.10\,\text{loc_norm}(v) + 0.35\,\text{complexity_norm}(v) \\ & + 0.30\,\text{instability_code}(v) + 0.25\,\text{lcom_norm}(v) \end{aligned} \tag{4}$$

$\text{CQP}(v) \in [0, 1]$ provides an explicit, quantitative bridge from source-code technical debt to architectural criticality, entering directly into the Maintainability dimension of the quality model.

### 3.4 Hierarchical Reliability–Maintainability (RM) Quality Model

Grounded in **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) (ISO/IEC, 2023a,b), SaG decomposes architectural quality into two primary dimensions—**Reliability** and **Maintainability**—providing explainable, role-specific diagnostics (Figure 2, Table 2).

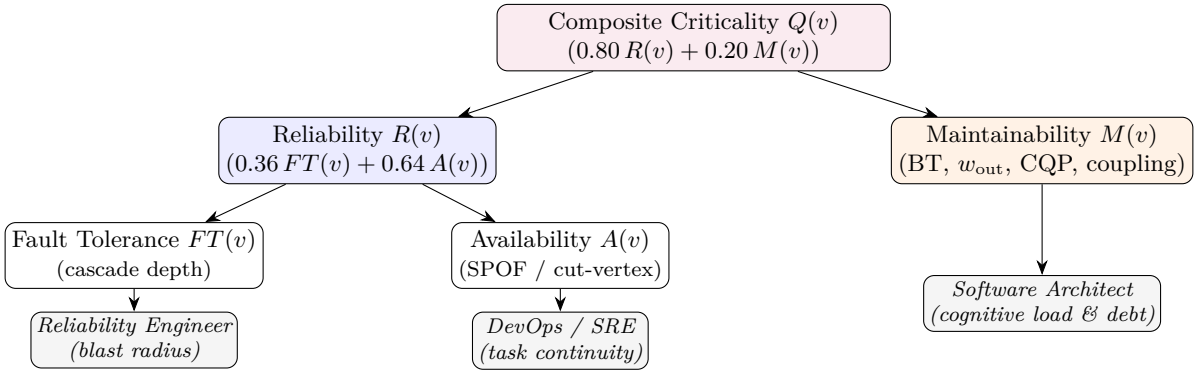



Figure 2: The hierarchical RM Diagnostic Attribution Model and stakeholder role mapping. Each arrow represents a deterministic scoring dependency computed in a single pass.

Table 2: RM Quality Dimensions, Stakeholder Roles, and ISO/IEC 25019 Quality-in-Use Mapping.

| Dim. | Sub-Characteristic | Engineering Role | High Score Meaning | ISO 25019 Harm |
|---|---|---|---|---|
| **R** | **Fault Tolerance** ($FT$) | Reliability Engineer | Cascades deeply; large blast radius | Indirect Harm |
| | **Availability** ($A$) | DevOps / SRE | Single point of failure; partition | Primary Task Loss |
| **M** | Structural Coupling | Software Architect | High change ripple; code-level debt | Maintainer Effort |

**Formal Quality Definitions.**

- **Definition D1 (Component Criticality):** Let $G_l = (V_l, E_l)$ be a layer-projected graph. Component criticality $\text{crit}_l : V_l \to [0,1]^2 \times [0,1]$ maps vertex $v$ to metric vector $\mathbf{s}(v) = [R(v), M(v)]^T$ and composite score $Q(v)$, estimating Quality-in-Use degradation.
- **Definition D2 (Relationship Criticality):** Let $e = (u, v) \in E_l$ be a dependency edge. Relationship criticality $\text{crit}_l(e) \in [0,1]$ estimates Quality-in-Use loss resulting from link disruption.
- **Definition D3 (Consequence Metric):** Criticality measures structural consequence given failure. Failure likelihood is supplied externally from operational telemetry.
- **Definition D4 (Relative Distribution):** Criticality classifications are relative to the score distribution of system $S$ at layer $l$.

**Structural Inputs.** Let $\varepsilon = 10^{-9}$. The model computes the following structural metrics on $G_l$:

- RPR($v$): PageRank on the transposed graph $G_l^{\mathsf{T}}$, rank-normalized over the population. Because `DEPENDS_ON` edges point dependent $\rightarrow$ dependency, failure propagates *against* edge direction; reverse PageRank is thus the true cascade-reach estimator.
- $\mathrm{DG}_{\mathrm{in}}(v) = \deg^-(v)/(|V_l|-1)$ and $\mathrm{DG}_{\mathrm{out}}(v) = \deg^+(v)/(|V_l|-1)$.
- **Multi-Path Coupling Index:** $\mathrm{MPCI}(v) = \sum_{e\in\mathrm{In}(v)} \max(\mathrm{paths}(e)-1,\,0)\,/\,(|V_l|-1)$, where paths($e$) counts distinct pub–sub channels realizing dependency $e$.
- **Fan-Out Criticality** (Topic vertices): $\mathrm{FOC}(t) = \ln(1+f(t))\,s(t)\,/\,\max_{t'}[\ln(1+f(t'))\,s(t')]$, for publication frequency $f(t)$ in Hz and subscriber count $s(t)$.
- **Directed Articulation Score:** $\mathrm{AP}_{c,\mathrm{directed}}(v) = \max(\mathrm{AP}_c^{\mathrm{out}}(v), \mathrm{AP}_c^{\mathrm{in}}(v))$, where $\mathrm{AP}_c^{\mathrm{out}}(v) = 1 - L(G_l \setminus \{v\})/(|V_l|-1)$ and $L(H)$ is the order of the largest connected component of $H$.
- **Bridge Ratio:** $\mathrm{BR}(v) = |\{e \in \mathrm{bridges}(G_l^{\mathrm{und}}) : v \in e\}|/\deg^{\mathrm{und}}(v)$.
- **Connectivity Degradation Index:** $\mathrm{CDI}(v) = \min\big([\bar{d}(G_l \setminus \{v\}) - \bar{d}(G_l)]/\bar{d}(G_l),\ 1\big)$, for $\bar{d}$ the average shortest-path length over reachable pairs.
- BT($v$): betweenness centrality; CC($v$): clustering coefficient; PC($v$): normalized path complexity; $w(v)$: normalized QoS component weight; $w_{\mathrm{out}}(v)$: normalized summed efferent QoS weight.

**Mathematical Formulation of Quality Dimensions.**

1. **Fault Tolerance ($FT$).** Estimates cascade reach. For Application, Library, Broker, and Node vertices, an enhanced cascade-depth potential CDPot discounts vertices that forward more than they absorb, amplified by multi-path coupling:

$$\begin{aligned} \mathrm{CDPot}_{\mathrm{base}}(v) &= \frac{\mathrm{RPR}(v)+\mathrm{DG}_{\mathrm{in}}(v)}{2}\left(1-\min\left(\frac{\deg^+(v)}{\max(\deg^-(v),\varepsilon)},\,1\right)\right) \\ \mathrm{CDPot}(v) &= \min\big(\mathrm{CDPot}_{\mathrm{base}}(v)\,(1+\mathrm{MPCI}(v)),\ 1\big) \\ FT(v) &= 0.45\,\mathrm{RPR}(v) + 0.30\,\mathrm{DG}_{\mathrm{in}}(v) + 0.25\,\mathrm{CDPot}(v) \end{aligned} \tag{5}$$

   For Topic vertices, the blast surface is the subscriber set moderated by publisher redundancy:

$$FT_{\mathrm{topic}}(t) = 0.50\,\mathrm{FOC}(t) + 0.50\,\mathrm{FOC}(t)\big(1-\min(w_{\mathrm{in}}(t),1)\big) \tag{6}$$

2. **Availability ($A$).** Evaluates single-point-of-failure exposure. The interaction term $\mathrm{QSPOF}(v) = \mathrm{AP}_{c,\mathrm{directed}}(v)\cdot w(v)$ deliberately incorporates QoS weight so structural SPOF risk is not masked at low weight:

$$\begin{aligned} A(v) = {} & 0.25\,\mathrm{AP}_{c,\mathrm{directed}}(v) + 0.20\,\mathrm{QSPOF}(v) \\ & + 0.20\,\mathrm{BR}(v) + 0.25\,\mathrm{CDI}(v) + 0.10\,w(v) \end{aligned} \tag{7}$$

3. **Maintainability ($M$).** Blends betweenness, efferent QoS out-degree, CQP, coupling risk, and inverse clustering. With $\iota(v) = \deg^+(v)/(\deg^-(v)+\deg^+(v)+\varepsilon)$:

$$\begin{aligned} \mathrm{CouplingRisk_enh}(v) &= \min\big(1,\ \big[1-|2\iota(v)-1|\big]\,(1+0.10\,\mathrm{PC}(v))\big) \\ M(v) &= 0.35\,\mathrm{BT}(v) + 0.30\,w_{\mathrm{out}}(v) + 0.15\,\mathrm{CQP}(v) \\ &\quad + 0.12\,\mathrm{CouplingRisk_enh}(v) + 0.08\,(1-\mathrm{CC}(v)) \end{aligned} \tag{8}$$

4. **Reliability Composite ($R$).** Blends Fault Tolerance and Availability with $\alpha = 0.36$:

$$R(v) = \alpha\cdot FT(v) + (1-\alpha)\cdot A(v) \tag{9}$$

**Composite Criticality and Stakeholder Projection.** Composite criticality blends Reliability and Maintainability:

$$Q(v) = 0.80\,R(v) + 0.20\,M(v) \tag{10}$$

The diagnostic vector $\mathbf{s}_{\mathrm{RM}}(v) = [R(v), M(v)]^T$ projects into ISO/IEC 25019 Quality-in-Use stakeholder harm scores $[H_{\mathrm{Ben}}, H_{\mathrm{Risk}}, H_{\mathrm{Acc}}]^T$ via transformation matrix $\mathbf{M}_{\mathrm{RM}\to\mathrm{QiU}}$:

$$\mathbf{h}_{\mathrm{QiU}}(v) = \mathbf{M}_{\mathrm{RM}\to\mathrm{QiU}} \cdot \mathbf{s}_{\mathrm{RM}}(v) = \begin{bmatrix} 0.75 & 0.25 \\ 0.80 & 0.20 \\ 0.60 & 0.40 \end{bmatrix} \begin{bmatrix} R(v) \\ M(v) \end{bmatrix} \tag{11}$$

$\mathbf{M}_{\mathrm{RM}\to\mathrm{QiU}}$ is row-stochastic, providing a declared re-presentation of RM in stakeholder-facing terms for routing findings to responsible engineering roles. Components are classified into five severity tiers (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`, `MINIMAL`) using adaptive box-plot fences ($Q > Q_3 + 1.5\,\mathrm{IQR}$ for `CRITICAL`; $Q_3 < Q \leq$ fence for `HIGH`).

### 3.5 Substrate Adequacy: Layer Requirements for Topological Metrics

A foundational insight of our metamodel is that architectural quality dimensions are not equally well-posed on all layer projections. Continuous metrics ($FT$ and $M$) vary smoothly across all layers. In contrast, discrete cut-vertex and bridge predicates ($\mathrm{AP}_{c,\mathrm{directed}}$ and BR in Availability) fire only where the projected graph contains cut vertices.

On densely interconnected application-layer projections where topics allow multiple redundant paths, cut structures can be structurally absent ($F_1 = 0.0$ for articulation-point detection). Because $A(v)$ carries $(1 - \alpha) = 0.64$ of $R(v)$'s weight, an availability score evaluated on an inadequate substrate collapses to a near-constant baseline. Substrate adequacy is therefore a design-time property: Availability-focused analysis (such as SPOF detection) must be evaluated on layer projections where physical or mediation bottlenecks exist—specifically the Middleware or System layers.

## 4 A Catalog of Architectural Anti-Patterns for Publish–Subscribe Systems

### 4.1 Architectural Anti-Patterns vs. Smells in Systems Modeling

In model-based systems engineering, structural quality findings fall into two primary confidence tiers:

- **Architectural Anti-Pattern:** A proven structural pathology known to cause systemic vulnerability or failure cascades under message bursts, requiring model-level architectural refactoring (e.g., Single Point of Failure, God Component, Broker Overload).
- **Architectural Bad Smell:** A localized topological anomaly signaling potential maintainability or reliability degradation, warranting architectural inspection (e.g., Chatty Pair, QoS Mismatch, Orphaned Topic).

Publish–subscribe architectural anomalies leave distinct **topological signatures** in the dependency graph. Because these signatures are directly computable from static architectural models, SaG detects them early in the design lifecycle before deployment on physical target clusters.

## 4.2 Detection Methodology

Detection operates over fourteen structural metrics derived from $G_{\text{analysis}}$: Reverse PageRank (RPR), in-degree and out-degree ($\text{DG}_{\text{in}}$, $\text{DG}_{\text{out}}$), multi-path coupling index (MPCI), topic fan-out criticality (FOC), betweenness centrality (BT), QoS-weighted in- and out-degree ($w_{\text{in}}$, $w_{\text{out}}$), clustering coefficient (CC), path complexity (PC), directed articulation score ($\text{AP}_{c,\text{directed}}$), bridge ratio (BR), connectivity degradation index (CDI), and component weight ($w$)—plus the synthesized Code Quality Penalty (CQP, Section 3.3). This yields fifteen continuous metrics evaluated against adaptive box-plot fences ($Q_3 + k\,\text{IQR}$, parameterized with $k = 1.5$).

## 4.3 Catalog Overview

Table 3 summarizes the nineteen formal anti-patterns across three severity tiers, four quality categories, and four finding scopes: **Component** (attributed to a single vertex), **Edge** (attributed to a directed relationship), **Pair** (attributed to a bidirectional component pair), and **System** (a global topology characteristic).

Table 3: Publish–Subscribe Architectural Anti-Pattern and Bad Smell Catalog. **Trig.** marks patterns that fire at least once on the eight-scenario benchmark corpus (Section 4.6).

| Pattern | Sev. | Category | Scope | Detection Signal | Autom. Refactoring | Trig. |
|---|---|---|---|---|---|---|
| `SPOF` | CRITICAL | Availability | Component | Articulation point $\wedge$ QoS SPOF score | Anti-affinity reallocation | ✓ |
| `SYSTEMIC_RISK` | CRITICAL | Reliability | System | Share of CRITICAL components $> 20\%$ | Advisory | |
| `CYCLE` | HIGH | Architecture | System | Strongly connected component / loop | Advisory | ✓ |
| `GOD_COMPONENT` | CRITICAL | Maintainability | Component | Extreme betweenness $\wedge$ CRITICAL $M(v)$ | Logical topic splitting | |
| `BOTTLENECK_EDGE` | HIGH | Availability | Edge | Edge betweenness centrality outlier | Logical topic splitting | ✓ |
| `BROKER_OVERLOAD` | HIGH | Availability | Component | Broker availability $\geq 2\times$ median load | Anti-affinity reallocation | |
| `DEEP_PIPELINE` | HIGH | Reliability | System | Path length $\geq \max(5, P_{75})$ | Advisory | (excl.) |
| `TOPIC_FANOUT` | MEDIUM | Reliability | Component | Topic subscriber out-degree outlier | Logical topic splitting | |
| `CHATTY_PAIR` | MEDIUM | Maintainability | Pair | Bidirectional weight product $> \tau_{\text{chatty}}$ | Advisory | |
| `QOS_MISMATCH` | MEDIUM | Reliability | Edge | Pub/Sub QoS-weight gap $> \tau_{\text{qos}}$ | Advisory (Manual / Relay) | ✓ |
| `ORPHANED_TOPIC` | MEDIUM | Maintainability | Component | Zero in- or out-degree on graph | Advisory | |
| `UNSTABLE_INTERFACE` | MEDIUM | Maintainability | Component | High `CouplingRisk_enh` $\wedge$ high $M(v)$ | Advisory | ✓ |
| `BRIDGE_EDGE` | HIGH | Availability | Edge | Graph-theoretic bridge cut-edge | Advisory | ✓ |
| `FAILURE_HUB` | CRITICAL | Reliability | Component | Reliability outlier $\wedge$ high out-degree | Logical topic splitting | |
| `CONCENTRATION_RISK` | MEDIUM | Reliability | System | Top-3 PageRank share $> 0.50$ | Advisory | |
| `HUB_AND_SPOKE` | MEDIUM | Maintainability | Component | Low clustering $\wedge$ degree $> 3$ | Logical topic splitting | ✓ |
| `CHAIN` | MEDIUM | Architecture | System | Degree-bounded linear subgraph | Advisory | |
| `ISOLATED` | MEDIUM | Architecture | Component | Zero total degree | Advisory | ✓ |
| `COMPOUND_RISK` | CRITICAL | Architecture | Component | Co-occurring SPOF + God/Hub finding | Anti-affinity reallocation | |

## 4.4 Representative Pattern Walkthroughs

1. **SPOF (Single Point of Failure):** A component whose removal disconnects the graph ($\text{AP}_{c,\text{directed}}(v) > 0$). Detection combines graph connectivity testing with QoS weighting

($\text{QSPOF}(v) = \text{AP}_c(v) \times w(v)$). Remediation generates host anti-affinity constraints.

2. **GOD_COMPONENT:** An application exhibiting extreme betweenness and CRITICAL maintainability ($\text{BT}(v) > 0.30 \wedge M(v) \in \text{CRITICAL}$). Remediation decomposes pub–sub channels via logical topic splitting.

3. **BROKER_OVERLOAD (Cautionary Benchmark Case):** A broker routing $\geq 2\times$ the median broker's traffic. *Cautionary finding:* In scenario S05 (two equally overloaded brokers), the within-population relative median rule cannot fire because each broker *is* the median, demonstrating the necessity of absolute capacity-aware thresholds.

4. **CHATTY_PAIR:** Two components maintaining bidirectional high-frequency dependencies across separate topics ($(u \to v) \wedge (v \to u)$), representing tight coupling masquerading as pub–sub decoupling.

5. **QOS_MISMATCH:** A publisher offering weaker reliability than the subscriber requires ($w_{\text{pub}}(u) < w_{\text{sub}}(v) - \tau$). In DDS/ROS 2, this produces silent connection drops at runtime without compile-time errors.

### 4.5 Detection Validation Methodology

Detection accuracy is validated against an independent discrete-event cascade simulation oracle ($I_{\text{comp}}(v)$) that models cascading message dropouts and queue exhaustion under component removal.

### 4.6 Catalog Coverage on the Benchmark Corpus

Across all eight benchmark scenarios, `DEEP_PIPELINE` is excluded due to path enumeration complexity (Section 7.7), and of the remaining eighteen active detectors, **eight trigger at least one finding** on this corpus (Table 3, **Trig.** column): Bottleneck Dependency (`BOTTLENECK_EDGE`, 4974 findings), QoS Policy Mismatch (`QOS_MISMATCH`, 1243), Unstable Interface (64), SPOF (17), Hub-and-Spoke (14), Dependency Cycle (`CYCLE`, 9), Bridge Edge (5), and Isolated Component (3). The remaining ten active patterns are fully unit-tested but require specific architectural topologies not present in these synthetic generators.

Crucially, of the 6329 total findings, only **98 (**1.5%**)** are directly component-scoped (SPOF 17, Unstable Interface 64, Isolated Component 3, Hub-and-Spoke 14); the remaining **6231 (**98.4%**)** are Edge- or System-scoped (dominated by Bottleneck Dependency and QoS Mismatches). This scope distribution explains the catalog's component-implication rate: edge-scoped findings implicate components as endpoints, resulting in broad screening sensitivity.

## 5 Tool Architecture and Implementation Techniques

### 5.1 Hexagonal Ports-and-Adapters Architecture and Dual-Repository Design

To decouple domain modeling and verification algorithms from underlying storage technologies, SaG is implemented according to a **hexagonal (ports-and-adapters) architecture** (Martin, 2003). As illustrated in Figure 3, core modeling services interact with graph storage exclusively through an abstract port interface, `IGraphRepository`.

The framework provides two specialized repository adapters, forming a **Dual-Repository Pattern**:

1. **`Neo4jRepository`:** A graph database adapter utilizing the official Bolt driver to communicate with a Neo4j cluster. This adapter supports declarative Cypher queries, persistent graph schemas, and interactive visual exploration during architectural review.

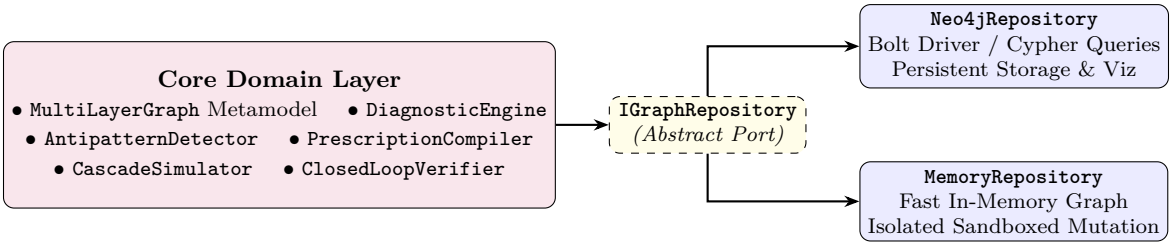



Figure 3: Hexagonal architecture of SaG: domain services access graph representations via the `IGraphRepository` port, supported by dual persistent and in-memory repository adapters.

2. **`MemoryRepository`:** A pure in-memory graph repository engineered in Python using vectorized adjacency tables and thread-safe copy-on-write semantics. While Neo4j excels at persistence, executing thousands of speculative graph mutations against an external database incurs prohibitive I/O and transaction serialization overhead. `MemoryRepository` allows the prescriptive engine to fork, mutate, and simulate counterfactual architectures in microseconds, enabling over 14,000 discrete-event simulation sweeps to complete in under 3 hours (Section 7.7).

## 5.2 The Seven Pipeline Stages

SaG operationalizes model analysis through a sequential, seven-stage processing pipeline:

1. **Stage 1 (Model Ingestion):** Ingests declarative Architecture-as-Code descriptors (JSON or YAML) and associated static code analysis metrics (LOC, WMC, LCOM, coupling instability).

2. **Stage 2 (Topological Projection):** Synthesizes the multi-layer graph $G$ and evaluates the seven derivation rules to generate the directed `DEPENDS_ON` analysis graph $G_{\text{analysis}}$.

3. **Stage 3 (Diagnostic Quality Attribution):** Computes CQP for application nodes, evaluates the fourteen structural graph metrics, derives $FT$, $A$, and $M$ quality scores, and projects them into ISO/IEC 25019 stakeholder harm vectors ($\mathbf{h}_{\text{QiU}}$).

4. **Stage 4 (Anti-Pattern Detection):** Executes the 19-pattern detection engine across Application, Middleware, Infrastructure, and System projections using adaptive box-plot fences.

5. **Stage 5 (Refactoring Candidate Compilation):** Compiles detected `CRITICAL` and `HIGH` findings into targeted candidate graph-mutation operators.

6. **Stage 6 (Counterfactual Verification):** Executes isolated, multi-seed cascade simulation sweeps within `MemoryRepository` sandboxes to filter out ineffective or noisy edits.

7. **Stage 7 (Presentation and Gating):** Emits verified architectural refactoring blueprints and CI/CD quality gate verdicts across standardized exit codes.

## 5.3 Graph-Mutation Refactoring Operators

The prescriptive engine implements three typed graph-mutation operators:

1. **Logical Topic Splitting (Operator 1):** Addresses `TOPIC_FANOUT`, `GOD_COMPONENT`, and `FAILURE_HUB`. This operator decomposes a high-congestion topic $T$ into dedicated sub-topics $\{T_a : a \in P(T)\}$, updating publisher and subscriber edge endpoints to isolate message streams and bound cascade reach.

2. **Physical Anti-Affinity Reallocation (Operator 2):** Addresses `SPOF`, `BROKER_OVERLOAD`, and `COMPOUND_RISK`. This operator migrates co-located critical components from a shared host node $N_{\text{from}}$ to an isolated host $N_{\text{to}}$, updating `RUNS_ON` and `HOSTED_ON` relationships to eliminate single-hardware failure domains.

3. **Transport QoS Contract Hardening (Operator 3):** Addresses volatile message channels operating under `BEST_EFFORT` or non-persistent contracts, updating transport specifications to `RELIABLE` and `TRANSIENT_LOCAL` with bounded history depths.

**Automation Footprint Disclosure.** Exactly **five of the nineteen patterns** directly trigger an automated mutation operator (`SPOF` → Operator 2; `GOD_COMPONENT`, `BOTTLENECK_EDGE`, `FAILURE_HUB`, `HUB_AND_SPOKE` → Operator 1). Operator 3 triggers from generic criticality tier membership. The remaining fourteen patterns provide advisory recommendations for human review. In practice, Operator 2 fires from RM criticality tier membership and `SPOF` findings, while Operator 1 fires on congestion, RM-critical topics, or bottleneck findings; Section 7.6 reports the resulting per-operator breakdown.

## 5.4 Closed-Loop Counterfactual Verification Engine

The prescriptive engine couples candidate compilation with closed-loop simulation verification. The idealized architectural optimization objective can be expressed as:

$$\min_{\Delta} \sum_{v \in V} I^*_{\Delta(G)}(v) \quad \text{subject to} \quad \text{Cost}(\Delta) \leq \mathcal{B} \tag{12}$$

where $\Delta(G)$ represents an architectural transformation producing mutated topology $G' = \Delta(G)$, and $I^*$ is the simulated failure impact.

Rather than executing an unconstrained search over the combinatorial space of graph transformations, SaG implements a **two-level verification-gated screening rule**:

1. **Per-Edit Counterfactual Acceptance Filter (Level 1):** Each candidate refactoring edit $\delta$ is applied in isolation to a sandboxed copy of $G$ inside `MemoryRepository`. The engine executes discrete-event failure cascade simulations across propagation thresholds $\Theta = \{0.1, 0.2, 0.5\}$ and random seeds $S = \{42, 123, 456\}$. An edit is retained if and only if its measured cascade-impact reduction exceeds simulator seed noise at every threshold:

$$\overline{\Delta I}_\theta(\delta) > \kappa \cdot \sigma_{\text{seed},\theta}(\delta) \quad \forall \theta \in \Theta \tag{13}$$

   where $\kappa = 1.0$ is the noise-rejection margin parameter.

2. **Whole-Policy Acceptance Gate (Level 2):** The surviving set of admitted edits $\Delta_{\text{admitted}}$ is applied jointly to $G$. The composite policy is accepted if and only if the net System Risk Index improves:

$$\Delta\text{SRI} = \text{SRI}_{\text{baseline}} - \text{SRI}_{\text{mutated}} > 0 \tag{14}$$

## 5.5 Substrate Independence and View Isolation

Candidate generation and per-edit verification are decoupled at the foundational modeling level: candidates are compiled from the projected dependency graph $G_{\text{analysis}}$ and diagnostic RM scores, while simulation verification is executed on the structural graph $G_{\text{structural}}$ within `MemoryRepository`. Consequently, the per-edit acceptance statistic $\overline{\Delta I}$ is evaluated by an independent discrete-event cascade simulator that shares no computational substrate with the RM score that proposed the candidate.

This independence does not extend to the second-level gate, where $\Delta$SRI is evaluated on RM dimensions ($\text{SRI} = 0.5\,\bar{R}_w + 0.5\,\bar{M}_w$). We therefore treat cascade impact reduction $\overline{\Delta I}_{\text{policy}}$ as the primary outcome measure and $\Delta$SRI as an internal consistency check (Section 7.6).

# 6 Tool Interfaces, Usability, and Engineering Ecosystem

## 6.1 Multi-Modal Tool Interfaces

A primary design goal of SaG is to support diverse engineering stakeholders across the software modeling lifecycle. To achieve this, the tool provides four synchronized interaction modalities, depicted in Figure 4.

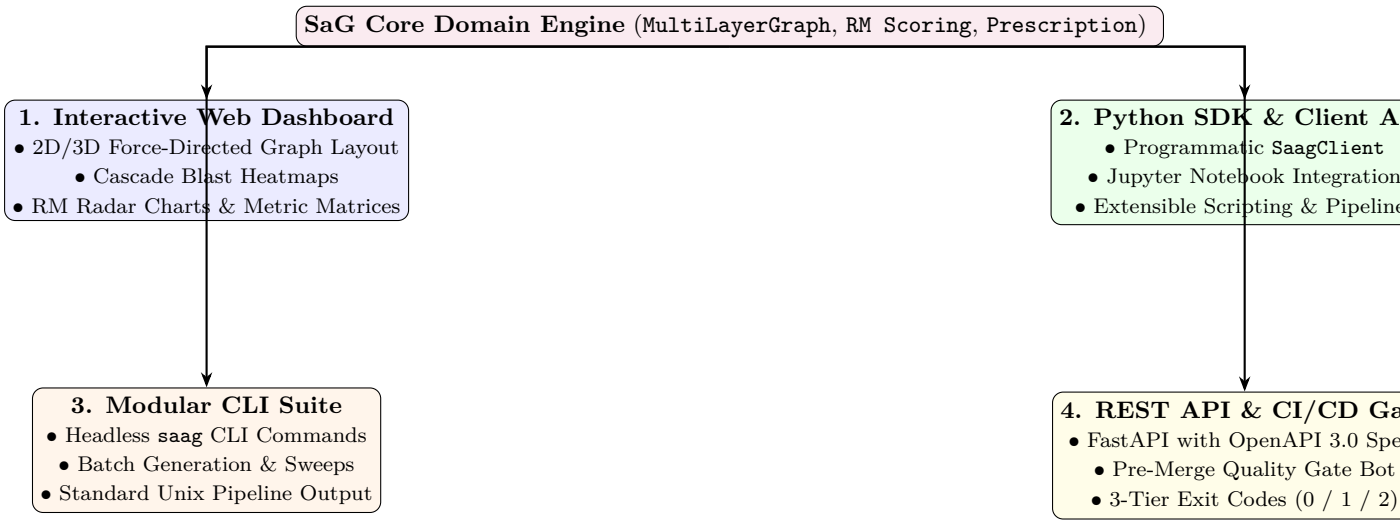



Figure 4: Multi-modal interaction ecosystem of the Software-as-a-Graph framework.

### 6.1.1 Interactive Web Dashboard and Visual Graph Explorer

For software architects and reliability engineers, SaG provides an interactive web dashboard developed in Next.js and React, complemented by a zero-dependency self-contained HTML dashboard generator (`saag.visualization.dashboard`). The visual explorer provides:

- **Dynamic Topology Visualization:** Interactive 2D/3D force-directed network graphs rendering nodes, topics, brokers, and dependencies with color-coded severity tiers.
- **Cascade Heatmaps:** Visual overlays depicting simulated blast radii under single- and multi-node failure injection.
- **RM Diagnostic Radar Charts:** Multi-axis radial plots comparing Fault Tolerance, Availability, and Maintainability across subsystems.
- **Dependency Matrix and Triage Views:** Tabular cross-layer dependency matrices highlighting high-coupling hotspots and anti-pattern findings.

### 6.1.2 Python SDK and Client API

For systems researchers and data scientists, SaG exposes a native Python client library (`saag.client.SaagClient`) and pipeline orchestration interface (`saag.pipeline.Pipeline`). The SDK enables programmatic topology generation, metric extraction, counterfactual model mutation, and statistical validation within Jupyter notebooks and automated simulation testbeds:

```
from saag.client import SaagClient
client = SaagClient.from_config("config/saag.yaml")
graph = client.model.import_topology("data/scenarios/s01_av.json")
diagnosis = client.diagnose.evaluate(graph, layer="app")
```

```
print(f"Critical components: {diagnosis.get_critical_nodes()}")
prescription = client.prescribe.run_verified(graph, margin=1.0)
```

### 6.1.3 Command-Line Interface (CLI) Suite

For automated batch execution and headless cluster sweeps, SaG offers a modular command-line suite:

- `saag model`: Ingests architecture manifests and populates graph repositories.
- `saag diagnose`: Computes CQP, RM attributions, and executes anti-pattern detectors.
- `saag prescribe`: Compiles and counterfactually verifies refactoring candidates.
- `saag simulate`: Runs discrete-event cascade simulations with configurable seed sweeps.
- `saag validate`: Correlates structural predictions against ground-truth cascade outcomes.

### 6.1.4 RESTful API Service

SaG includes a production-ready asynchronous web service built with FastAPI (`api/main.py`). The API exposes full OpenAPI 3.0 documentation, enabling external engineering platforms (e.g., Jenkins, GitLab CI, GitHub Actions) to trigger analysis and retrieve serialized JSON diagnostic payloads over HTTP.

## 6.2 DevOps Integration and CI/CD Quality Gating

To prevent architectural regression during continuous system evolution, SaG's diagnostic engine is packaged as an automated quality gate for continuous integration workflows.

### 6.2.1 Regression Semantics: Absolute vs. Delta-Aware Gating

The gate supports two operational evaluation modes:

- **Absolute Gating:** Evaluates the complete system topology against global thresholds. However, mature industrial software systems frequently maintain accepted, intentional architectural debt. Under absolute gating on our benchmark corpus, the presence of legacy `CRITICAL` or `HIGH` findings triggers **Exit Code 2 (Block) on all eight scenarios** (Section 7.7). This demonstrates that absolute gating is too rigid for iterative development.
- **Delta-Aware Gating (Design):** Compares the candidate pull-request topology against the target branch merge-base, blocking only *newly introduced* anti-patterns or worsening severity classifications, while respecting signed **waiver registers** for risk-accepted legacy debt.

### 6.2.2 Three-Tier Exit-Code Protocol

The quality gate communicates verdicts to CI runners via a standardized three-tier exit-code protocol:

- **Exit Code 0 (Pass):** No critical architectural anti-patterns detected; build is cleared for deployment.
- **Exit Code 1 (Warning):** Non-critical `MEDIUM`-severity smells detected; build passes with advisory warnings posted to developers.
- **Exit Code 2 (Block):** Severe `CRITICAL` or `HIGH` anti-patterns detected; deployment pipeline is halted.

# 7 Empirical Application in Realistic Scenarios

## 7.1 Benchmark Scenarios and Real-World Corpora

To evaluate SaG across diverse system domains and scales, we employ an evaluation corpus comprising eight synthetic benchmark scenarios (Table 4) and three architectures transcribed from open-source systems.

Table 4: Benchmark Scenario Scales and Topological Dimensions.

| Scenario | Applications | Libraries | Topics | Brokers | Nodes | Total Edges ($\|E\|$) |
|---|---|---|---|---|---|---|
| S01 Autonomous Vehicle | 80 | 20 | 40 | 4 | 8 | 797 |
| S02 IoT Smart City | 200 | 10 | 80 | 6 | 30 | 1322 |
| S03 Financial Trading | 60 | 18 | 35 | 5 | 6 | 580 |
| S04 Healthcare | 50 | 12 | 25 | 3 | 8 | 400 |
| S05 Hub-and-Spoke | 70 | 25 | 30 | 2 | 12 | 797 |
| S06 Microservices Mesh | 90 | 30 | 45 | 6 | 15 | 680 |
| S07 Hyper-Scale Enterprise | 300 | 50 | 120 | 10 | 40 | 3245 |
| S08 Tiny Regression | 12 | 4 | 8 | 2 | 3 | 101 |
| **Transcribed Real-World Topologies** | | | | | | |
| Autoware.Universe (ROS 2/DDS) | 32 | 10 | 24 | 3 | 6 | 284 |
| Train-Ticket (Microservices) | 41 | 8 | 30 | 3 | 8 | 342 |
| Cloud IoT Mesh (MQTT) | 22 | 8 | 20 | 4 | 6 | 195 |

Scenarios S01–S08 span six architectural domains ranging from 29 to 520 vertices (12 to 300 applications). Scenarios S01–S07 evaluate prescriptive refactoring; S08 is retained as a regression smoke test. To evaluate real-world validity, we include three architectures transcribed from production open-source systems: **Autoware.Universe** (the leading open-source ROS 2/DDS autonomous driving stack), the **Train-Ticket** microservices benchmark, and a **Cloud IoT Mesh** modeled on an industrial e-commerce and MQTT telemetry gateway.

## 7.2 Research Questions and Evaluation Protocol

Our empirical evaluation addresses five central research questions:

- **RQ1 (Diagnostic Ranking Efficacy):** How does the RM composite's rank correlation with cascade failure impact compare to structural baselines (betweenness, degree centrality)?
- **RQ2 (Catalog Discrimination and Calibration):** How does the anti-pattern catalog perform on synthetic versus real-world topologies, and what explains its implication rate?
- **RQ3 (Attribution Validity and Substrate Adequacy):** Does the Availability dimension carry genuine signal on the application layer, and how does node stratification resolve Simpson's paradox?
- **RQ4 (Prescriptive Refactoring Efficacy and Operator Survival):** Does the closed-loop prescriptive engine achieve measurable reductions in cascade impact and System Risk Index, and how do individual mutation operators survive counterfactual verification?
- **RQ5 (Tool Performance and Verification Latency):** What is the execution latency of diagnostic gating and counterfactual verification across system scales?

**Evaluation Protocol and Metrics.**

- **Detection Metrics:** Spearman rank correlation $\rho(Q, I_{\text{comp}})$, Cohen's $\kappa$, Normalized Discounted Cumulative Gain (NDCG@10), Top-10 overlap, precision, recall, and $F_1$ against discrete-event simulation ground truth $I_{\text{comp}}(v)$.
- **Prescriptive Metrics (Primary):** Mean fractional cascade-impact reduction on the fully mutated graph: $\overline{\Delta I}_{\text{policy}} = \text{mean}_v\,[I(v) - I'(v)]/I(v)$, measured by the independent cascade simulator.
- **Prescriptive Metrics (Secondary):** System Risk Index change $\Delta\text{SRI} = \text{SRI}_{\text{baseline}} - \text{SRI}_{\text{mutated}}$, serving as an internal consistency check.
- **Verification Parameters:** Propagation thresholds $\Theta = \{0.1, 0.2, 0.5\}$, random seeds $S = \{42, 123, 456\}$, and noise rejection margin $\kappa = 1.0$.

### 7.3 Diagnostic Ranking Efficacy and Predictor Saturation (RQ1)

Table 5 summarizes diagnostic performance at the application layer across all eight scenarios. Table 6 compares the RM composite against single-metric baselines.

Table 5: Diagnostic detection validation (app layer) against simulation ground truth $I_{\text{comp}}(v)$.

| **Scenario** | $n$ | $\rho(Q, I)$ | $F_1$ | **Top-5** | **Catalog P** | **Catalog R** | **Implicated %** | **Gate (s)** |
|---|---|---|---|---|---|---|---|---|
| S01 Autonomous Vehicle | 80 | 0.457 | 0.600 | 0.40 | 0.244 | 0.950 | 97.5% | 0.59 |
| S02 IoT Smart City | 200 | 0.718 | 0.780 | 0.40 | 0.310 | 0.880 | 71.0% | 0.56 |
| S03 Financial Trading | 60 | 0.365 | 0.667 | 1.00 | 0.237 | 0.933 | 98.3% | 0.40 |
| S04 Healthcare | 50 | 0.451 | 0.538 | 0.60 | 0.213 | 0.769 | 94.0% | 0.16 |
| S05 Hub-and-Spoke | 70 | 0.466 | 0.611 | 0.80 | 0.257 | 1.000 | 100.0% | 0.92 |
| S06 Microservices Mesh | 90 | 0.405 | 0.565 | 0.40 | 0.261 | 1.000 | 97.8% | 0.35 |
| S07 Hyper-Scale Enterprise | 300 | 0.480 | 0.627 | 0.20 | 0.251 | 1.000 | 99.7% | 20.98 |
| S08 Tiny Regression | 12 | 0.538 | 0.333 | 0.60 | 0.250 | 1.000 | 100.0% | 0.01 |
| **Mean** | — | **0.485** | **0.590** | **0.550** | **0.253** | **0.942** | **94.8%** | **3.00** |

Table 6: Predictor Comparison (App Layer, Mean over 8 Scenarios). Structural predictors saturate around comparable ranking quality.

| **Predictor** | $\rho$ | **Cohen's $\kappa$** | **NDCG@10** | **Top-10** |
|---|---|---|---|---|
| $Q(v)$ composite (RM) | 0.4850 | 0.4514 | 0.8295 | 0.5875 |
| Betweenness | 0.4346 | 0.4139 | **0.8592** | **0.6250** |
| Degree Centrality | **0.5187** | **0.4792** | 0.8214 | 0.5625 |

**Predictor Saturation.** As Table 6 illustrates, all three predictors saturate around comparable performance: degree centrality leads slightly on $\rho$ (0.519) and $\kappa$ (0.479), betweenness leads on NDCG@10 (0.859) and Top-10 overlap (0.625), while the RM composite $Q(v)$ achieves balanced ranking ($\rho = 0.485$, NDCG@10 = 0.830). Under a Wilcoxon paired signed-rank test across scenarios, $Q(v)$ versus degree centrality yields $p = 0.945$ (paired $t$-test, $p = 0.465$); head-to-head, $Q(v)$ attains higher correlation in **five of eight** scenarios. This empirical saturation demonstrates that no single scalar predictor dominates. Consequently, the primary value of the RM model is not superior ranking accuracy, but rather its **deterministic, role-attributed quality decomposition** into actionable Fault Tolerance, Availability, and Maintainability dimensions.

## 7.4 Catalog Calibration on Synthetic vs. Transcribed Topologies (RQ2)

On the generated benchmark scenarios, the anti-pattern catalog exhibits high screening recall (0.942) and low precision (0.253), implicating an average of 94.8% of components with chance-level agreement ($\kappa = -0.0022$). As detailed in Section 4.6, this is driven by finding scope: 98.4% of findings are edge-scoped (`BOTTLENECK_EDGE` and `QOS_MISMATCH`), which implicate all connected endpoints.

To assess whether this is an artifact of synthetic topology generation, we evaluated the catalog across the three transcribed real-world open-source systems over five simulation seeds (Table 7).

Table 7: Catalog behavior on generated versus transcribed topologies (System Layer, 5 seeds). The catalog discriminates effectively on real-world architectures.

| **Measure** | **Generated** ($n = 40$) | **Transcribed** ($n = 15$) |
|---|---|---|
| Cohen's $\kappa$ | $-0.036 \pm 0.049$ | $\mathbf{+0.296 \pm 0.100}$ |
| Precision | $0.237 \pm 0.014$ | $\mathbf{0.402 \pm 0.060}$ |
| Recall | $0.887 \pm 0.059$ | $0.861 \pm 0.105$ |
| $F_1$ | $0.374 \pm 0.022$ | $\mathbf{0.545 \pm 0.061}$ |
| Components Implicated | $93.7\% \pm 3.2$ | $\mathbf{56.3\% \pm 9.8}$ |

On transcribed architectures, chance-corrected agreement rises to $\kappa = \mathbf{+0.296}$ (fair agreement), precision increases by two-thirds to 0.402, and the component implication rate drops from 93.7% to 56.3% (21.3% in Autoware, 18.3% in Cloud Mesh, 14.4% in Train-Ticket). This confirms that the catalog's indiscriminate behavior is largely an artifact of the dense uniform edge distributions in synthetic generators, whereas modular real-world architectures provide natural structural boundaries where anti-patterns discriminate effectively.

## 7.5 Attribution Validity and Substrate Adequacy (RQ3)

**Availability Degeneracy on Application Layer.** At the application layer, the `SPOF` detector scores $F_1 = 0.0$ in **all eight scenarios**, predicting zero articulation points in six scenarios. Because $A(v)$ carries 0.64 of $R(v)$'s weight, half of the composite score on this layer is computed over a near-constant substrate. This confirms Section 3.5's substrate adequacy theorem: Availability metrics must be evaluated on Middleware or System layer projections where brokers and compute nodes create genuine physical bottlenecks.

**Reconciling Prior Conference Findings.** Our preliminary conference study (Yigit and Buzluca, 2025) reported $\rho = 0.94$ on a 10-application system validated against reachability loss. That earlier result shared its computational substrate with the predictor (both computed on the undirected application graph) and examined a topology with deliberate cut-vertices. This paper evaluates an independent discrete-event cascade oracle on directed dependency graphs across up to 300 applications, revealing the realistic saturation baseline ($\rho \approx 0.485$).

**Simpson's Paradox in Multi-Type Pooling.** When all five entity types are pooled at the system layer, the aggregate correlation collapses to $\rho = 0.028$, even though within-type strata show strong positive correlation (Application 0.503, Broker 0.395, Node 0.142). This is a textbook instance of Simpson's paradox arising from baseline score offsets between entity types, demonstrating that stratified evaluation is a mandatory requirement for heterogeneous software graphs.

## 7.6 Prescriptive Refactoring Performance and Operator Survival (RQ4)

Table 8 details the closed-loop refactoring results across all seven prescriptive scenarios, and Table 9 breaks down admission by mutation operator.

Table 8: Prescriptive refactoring performance under per-edit counterfactual verification.

| **Scenario** | $\overline{\Delta I}_{\text{policy}}$ (primary, oracle) | $\Delta$SRI (secondary, RM) | **Cand.** | **Adm.** | **Rej.** | **Adm. %** |
|---|---|---|---|---|---|---|
| S01 Autonomous Vehicle | +0.0161 | +0.0086 | 156 | 45 | 111 | 28.8% |
| S02 IoT Smart City | +0.0822 | **+0.0461** | 391 | 297 | 94 | 76.0% |
| S03 Financial Trading | **−0.0689** | +0.0108 | 125 | 22 | 103 | 17.6% |
| S04 Healthcare | **+0.1894** | +0.0109 | 99 | 36 | 63 | 36.4% |
| S05 Hub-and-Spoke | +0.0236 | +0.0115 | 127 | 107 | 20 | 84.3% |
| S06 Microservices Mesh | **−0.0015** | +0.0097 | 163 | 142 | 21 | 87.1% |
| S07 Hyper-Scale Enterprise | +0.0060 | **+0.0171** | 528 | 479 | 49 | 90.7% |
| **Mean / Total** | **+0.0352** | — | **1589** | **1128** | **461** | **71.0%** |
| **Scenarios Improved** | **5 / 7** | **7 / 7** | | | | |

Table 9: Candidate Generation vs. Verified Admission by Refactoring Operator.

| **Refactoring Operator** | **Candidates** | **Admitted** | **Survival Rate** | **Primary Structural Target** |
|---|---|---|---|---|
| **Physical Anti-Affinity Reallocation** | 1188 | **923** | **77.7%** | Single points of failure, co-location |
| **Transport QoS Contract Hardening** | 69 | **40** | **58.0%** | Volatile transport contracts on topics |
| **Logical Topic Splitting** | 332 | **165** | **49.7%** | High-fan-out message channels, hubs |
| **Total** | **1589** | **1128** | **71.0%** | — |

**Key Prescriptive Findings.**

1. **Verification Yield:** Across 1589 compiled candidates, **1128 edits (**71.0%**)** survive counterfactual verification, while 461 edits (29.0%) are filtered out because they fail to clear simulator noise.

2. **Operator Survival:** Physical anti-affinity reallocation demonstrates the highest survival rate (77.7%), followed by transport QoS hardening (58.0%) and topic splitting (49.7%).

3. **Oracle-Measured Impact Reduction:** Under the independent cascade oracle, five of seven scenarios achieve net cascade-impact reduction (mean +3.52%, reaching +18.9% in Healthcare). Two scenarios regress slightly under joint composition (S03 −6.9%, S06 −0.15%), demonstrating that individually verified edits can interact sub-additively.

4. **Internal Risk Index ($\Delta$SRI):** All seven scenarios achieve positive $\Delta$SRI gains (Wilcoxon $W = 0, p = 0.0156$). However, because $\Delta$SRI evaluates the same RM dimensions used to compile candidates, we treat the independent oracle's +3.52% impact reduction as the definitive headline measure.

## 7.7 Tool Performance, Scalability, and Latency (RQ5)

1. **Diagnostic Pipeline Execution Latency:** The complete diagnostic pipeline (graph parsing, CQP computation, RM attribution, and 18 active detectors) executes in **0.01 s to**

**20.98 s** across the corpus (Table 5). The 18 detectors run in under **0.2 s total**; overhead is dominated by base shortest-path and centrality calculations.

2. `DEEP_PIPELINE` **Bounding:** Unconstrained path enumeration in the `DEEP_PIPELINE` detector generated 247,761 paths on small fixtures and hung on enterprise graphs. Excluding it restored sub-second execution across small-to-medium scenarios.

3. **Prescriptive Verification Throughput:** The complete prescriptive sweep across seven scenarios (comprising 14,301 discrete-event simulation sweeps) executed in **2 h 47 min** on a 20-core workstation using `MemoryRepository`, confirming the computational feasibility of nightly automated refactoring verification.

# 8 Community Importance and Lessons Learnt

In accordance with the guidelines of the SoSyM Tool Paper Call, this section explicitly articulates why the Software-as-a-Graph framework is important for the systems and modeling community, followed by five transferable lessons learnt from designing, building, testing, evaluating, and disseminating the tool.

## 8.1 Importance for the Systems and Software Modeling Community

1. **Addressing the MDE Blind Spot for Asynchronous Pub–Sub Systems:** While Model-Driven Engineering has produced mature modeling languages and tools for synchronous RPC, component-connector monoliths, and embedded control loops (e.g., AADL, Palladio, SysML), asynchronous publish–subscribe architectures have remained a significant blind spot. Contemporary cyber-physical systems, autonomous vehicles (ROS 2/DDS), and IoT clouds rely heavily on topic-based pub–sub middleware. SaG provides the modeling community with a formalized, open-source metamodel, multi-layer projection rules, and quality attribution framework tailored specifically to the semantics of asynchronous message-passing systems.

2. **A Standardized, Reproducible Experimental Benchmark Testbed:** A recurring obstacle in software architecture and modeling research is the scarcity of open, diverse, and realistic benchmark models. SaG delivers an open corpus of eight multi-scale benchmark topologies (spanning autonomous driving, smart cities, financial trading, healthcare, and microservices) alongside three transcribed real-world open-source architectures (Autoware.Universe, Train-Ticket, Cloud IoT Mesh). This corpus, complete with declarative descriptors and discrete-event simulation oracles, establishes a shared experimental substrate for researchers developing new graph metrics, smell detectors, and model optimizers.

3. **Reusable Catalog of Anti-Patterns and Refactoring Operators:** SaG formalizes nineteen architectural anti-patterns and three graph-mutation operators stated over native publish–subscribe constructs. By publishing complete metric formulas, detection algorithms, and mutation preconditions, SaG provides a modular specification that other MDE researchers can adopt, extend, or transcribe into other modeling formalisms (e.g., EMF/Ecore, AGG, Henshin).

4. **Operationalizing International Quality Standards in Graph Models:** Many predictive modeling tools rely on ad-hoc scalar metrics or opaque neural networks. SaG demonstrates how international quality standards (**ISO/IEC 25010** and **ISO/IEC 25019**) can be operationalized deterministically over graph models, establishing an auditable bridge between source-code metrics (CQP) and macro-architectural risk.

## 8.2 Lessons Learnt Designing, Building, Testing, and Disseminating the Tool

### 8.2.1 Lesson 1 (Designing): Metamodel Substrate Adequacy and Layer Awareness

*Architectural quality metrics are only as valid as the projection substrate on which they are evaluated.*

A common design instinct in MDE is to define a structural metric (such as articulation-point membership for Single Points of Failure) and evaluate it across whatever graph model is available. In SaG, our initial design evaluated the complete RM composite at the application layer. However, because application-layer projections abstract away underlying middleware brokers and host nodes, topics often present redundant alternate paths, rendering cut-vertices structurally nonexistent ($F_1 = 0.0$ for SPOF detection across all eight scenarios).

**Takeaway for tool designers:** Multi-level system models must formally specify the exact architectural layer on which each quality metric is mathematically well-posed. Discrete connectivity metrics must be evaluated on layers where physical mediation bottlenecks actually exist (e.g., Middleware and Infrastructure layers), rather than assuming that application-layer projections reflect global availability risks.

### 8.2.2 Lesson 2 (Building): Dual Repositories for Counterfactual Model Transformation

*Graph databases are ideal for persistent exploration, but in-memory copy-on-write sandboxes are mandatory for high-throughput model verification.*

Initially, our tool prototype executed all graph operations directly against Neo4j via Cypher queries. While Neo4j provided excellent persistence, expressive graph queries, and visual debugging, using it for closed-loop refactoring verification created severe performance bottlenecks: executing thousands of speculative graph mutations, running cascade simulations, and rolling back state incurred severe disk I/O, network serialization, and transaction locking delays.

**Takeaway for tool designers:** By adopting a hexagonal ports-and-adapters architecture around an abstract `IGraphRepository` port, we implemented a specialized `MemoryRepository`. This in-memory adapter uses vectorized adjacency lists and thread-safe copy-on-write branching, enabling isolated counterfactual mutation sweeps to execute in microseconds and completing 14,301 simulation sweeps in 2 h 47 min. Tool builders should cleanly separate persistent query stores from ephemeral, in-memory execution sandboxes.

### 8.2.3 Lesson 3 (Testing): The "Verify-Before-Transform" Discipline

*Rule-based model refactoring cannot be trusted open-loop; candidate transformations must be counterfactually verified against dynamic failure cascades.*

In the model refactoring literature, transformation rules are typically assumed to improve system quality by definition (e.g., splitting a congested topic must reduce coupling). When we tested this assumption against our discrete-event cascade simulator, we discovered that **461 of 1589 compiled refactoring candidates** (29.0%) failed verification: their measured cascade-impact reduction did not exceed stochastic seed noise, or they introduced subtle downstream congestion that exacerbated cascade reach. Furthermore, in two scenarios (S03 and S06), individually beneficial edits composed sub-additively into a net regression under joint application.

**Takeaway for tool designers:** In complex distributed architectures, static heuristic rules are insufficient to guarantee resilience. Refactoring tools must incorporate closed-loop counterfactual simulation or formal verification into the transformation pipeline, actively discarding plausible-looking edits that fail to demonstrate measurable dependability gains beyond noise.

#### 8.2.4 Lesson 4 (Evaluating): Explanatory Attribution vs. Scalar Centrality Ranking

*Multi-metric composite scores do not necessarily out-rank simple centralities; their true value lies in explainable, role-specific quality attribution.*

A foundational hypothesis in software metrics research is that multi-dimensional composite scores should significantly outperform simple graph centralities for predicting structural failure impact. Our empirical findings decisively qualify this assumption: across eight scenarios, degree centrality ($\rho = 0.519$), betweenness (NDCG@10 $= 0.859$), and the RM composite ($\rho = 0.485$, NDCG@10 $= 0.830$) saturate at statistically indistinguishable ranking quality (paired $p = 0.95$).

**Takeaway for tool designers:** Modeling researchers should resist the urge to claim superior ranking accuracy where empirical evidence demonstrates saturation. Instead, the true value of domain-grounded modeling is *explanatory attribution*: a scalar degree centrality cannot tell an engineer whether a component is critical due to deep cascade reach (Reliability Engineer), single-point-of-failure exposure (SRE), or high coupling debt (Software Architect). Grounding models in standardized, auditable quality taxonomies provides actionable engineering value that raw centralities cannot offer.

#### 8.2.5 Lesson 5 (Disseminating): Multi-Modal Ergonomics for Diverse Modeling Personas

*Tool adoption requires meeting diverse stakeholders in their native operational environments.*

During early dissemination of SaG, we observed that different engineering personas rejected tools that did not fit their established workflows. Systems architects preferred visual graph layouts, interactive heatmaps, and radar charts over raw data tables. Systems researchers and MDE modelers demanded scriptable Python APIs and Jupyter notebook integration for rapid experimentation. Conversely, DevOps engineers flatly rejected graphical interfaces, demanding headless CLI binaries with deterministic Unix exit codes (0/1/2) that could run within sub-second pre-merge CI/CD quality gates.

**Takeaway for tool designers:** A modeling tool must provide synchronized multi-modal interaction interfaces: an interactive web dashboard for visual exploration, a programmatic Python SDK for research, and a headless CLI and REST API for continuous automated governance.

# 9 Discussion and Tool Availability

## 9.1 Threats to Validity

- **Construct Validity — Circularity of Whole-Policy Gating:** While the Level 1 per-edit filter utilizes an independent cascade oracle, the Level 2 whole-policy gate evaluates ΔSRI, which reweights the same RM dimensions used to compile candidates. This is why Section 7.6 treats the oracle-measured cascade reduction ($\overline{\Delta I}_{\text{policy}}$) as the primary outcome and ΔSRI as an internal consistency check.

- **Construct Validity — Substrate Adequacy:** As formalized in Section 3.5, discrete topological predicates (e.g., articulation points) collapse on projections lacking cut structures. Availability metrics must be evaluated on Middleware or Infrastructure projections where mediation bottlenecks are physically present.

- **Internal Validity — Independence in Verification:** Decoupled in-memory graph sandboxing (`MemoryRepository`) guarantees that speculative graph mutations do not leak state into baseline graphs or cross-contaminate concurrent verification sweeps.

- **External Validity — Synthetic vs. Real-World Generalization:** The eight core benchmark scenarios are synthetic generators. We mitigated this by evaluating three

transcribed real-world open-source systems (Autoware.Universe, Train-Ticket, Cloud IoT Mesh), demonstrating that the catalog achieves substantial chance-corrected agreement ($\kappa = +0.296$) on realistic topologies.

- **Conclusion Validity — Sample Size and Effect Significance:** The primary oracle-measured reduction (+3.52%) across seven prescriptive scenarios does not achieve statistical significance under a two-sided Wilcoxon signed-rank test ($p = 0.219, n = 7$). We report this effect descriptively and provide complete per-scenario distributions.

### 9.2 Tool Availability, Reproducibility, and Open Science

In strict compliance with SoSyM's tool availability criteria, Software-as-a-Graph (SaG) is developed and maintained as an open-source, fully reproducible scientific software framework:

- **Open-Source License:** SaG is released under the permissive **Apache License 2.0**, granting complete freedom for academic research, extension, and industrial adoption.
- **Source Code Repository:** The full source code—including the core 'saag' domain engine, the 'cli' suite, the 'api' service, and the web visualization components—is publicly hosted on GitHub at `https://github.com/software-as-a-graph/software-as-a-graph`.
- **Persistent Archival and DOI:** The repository, benchmark scenario datasets, and experimental outputs are permanently archived on Zenodo with a persistent Digital Object Identifier (DOI), registered via `.zenodo.json`.
- **Package Distribution and Containerization:** SaG is packaged for Python $\geq 3.10$ via `pyproject.toml` and can be installed directly with standard package managers. For zero-configuration deployment, a complete multi-container environment is provided via `docker-compose.yml`, orchestrating the Neo4j 5.x graph database, the FastAPI backend, and the interactive web explorer.
- **Reproduction Package:** The repository contains an automated reproduction suite under `reproduce/`. Running the automated harness reproduces all numerical values, tables, and figures reported in Section 7 from raw artifact JSONs.

## 10 Conclusion and Future Work

### 10.1 Conclusion

This tool paper presented **Software-as-a-Graph (SaG)**, an open-source, extensible Model-Driven Engineering framework for graph-based architectural modeling, diagnostic quality analysis, and counterfactually verified refactoring of distributed publish–subscribe software systems. By formalizing a domain-specific, typed multigraph metamodel $(V, E, \tau_V, \tau_E, w_V, w_E)$ across four architectural layers, SaG bridges source-level code metrics (CQP) with macro-architectural risk, evaluating system health against ISO/IEC 25010 and ISO/IEC 25019 standards and formalizing nineteen publish–subscribe architectural anti-patterns.

To remediate architectural technical debt, SaG implements a closed-loop prescriptive engine driven by three graph-mutation operators. Adopting a strict **"verify-before-transform"** discipline, the tool simulates candidate mutations within in-memory sandboxes against a discrete-event cascade oracle, admitting only edits whose risk reduction clears stochastic seed noise. SaG's hexagonal architecture features a dual-repository pattern that pairs persistent Neo4j storage with an ultra-fast in-memory repository (`MemoryRepository`), supporting multi-modal interfaces spanning interactive web dashboards, Python SDKs, modular CLIs, REST APIs, and automated CI/CD quality gates.

Evaluated across eight benchmark scenarios and three transcribed real-world open-source systems (Autoware.Universe, Train-Ticket, Cloud IoT Mesh), SaG demonstrates high diagnostic screening sensitivity, superior discrimination on real-world modular architectures ($\kappa = +0.296$), a 71.0% refactoring admission rate, and sub-second diagnostic gating latency. Crucially, we synthesize five transferable lessons learnt spanning metamodel substrate adequacy, in-memory sandboxing, simulation-in-the-loop verification, explanatory attribution versus ranking saturation, and multi-modal tool ergonomics to inform future tool builders in the systems modeling community.

### 10.2 Future Work

We identify several promising avenues for future tool evolution:

1. **Layer-Aware Automated Availability Metrics:** Implementing multi-layer path analysis to automatically evaluate Availability predicates on Middleware or Infrastructure projections where cut structures are physically present.

2. **Search-Based Multi-Objective Optimization (SBSE):** Expanding the current rule-based candidate compiler into a hybrid multi-objective search algorithm (e.g., NSGA-II) that optimizes failure impact, hardware migration cost, and network bandwidth under budget constraints.

3. **Combinatorial Subset Verification:** Replacing the current greedy all-admitted policy gate with forward-selection subset verification to prevent sub-additive compositional regressions.

4. **External Tool Benchmarking:** Executing comparative evaluations against general-purpose architectural smell detectors (e.g., Arcan, Designite) running over projected dependency graphs.

5. **Automated Model Ingestion from IDLs and Traces:** Developing automated model extractors that directly parse ROS 2 package manifests, DDS XML QoS profiles, and Kafka topic definitions into SaG multigraphs.

6. **LLM-Assisted Refactoring Implementation:** Integrating large language models with SaG's verified graph mutation blueprints to automatically synthesize infrastructure-as-code diffs (Kubernetes manifests, Docker Compose files) for immediate deployment.

## Declarations

**Funding:** This work was supported in part by the Scientific and Technological Research Council of Türkiye (TÜBİTAK), grant number [GRANT-NUMBER].

**Competing Interests:** The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to the work submitted for publication.

**Related Submissions and Overlap Disclosure:** A companion manuscript by the same authors, *Software-as-a-Graph: A Static System Analysis Framework for Pre-Deployment Quality Gating and Failure Simulation of Publish-Subscribe Middleware*, is currently under review at another journal. The two submissions share three research artifacts—the heterogeneous graph schema, the benchmark scenario corpus, and the discrete-event cascade simulator—and are otherwise disjoint. That manuscript contributes the learned Graph Neural Network pathway for failure-impact prediction and its evaluation; this manuscript contributes the deterministic

Diagnostic Pathway, the Code Quality Penalty, the nineteen-pattern anti-pattern catalog, the prescriptive refactoring compiler, the counterfactual verification engine, and the CI/CD quality gate. No result, table, or figure appears in both. We disclose this so the editor can assess the overlap directly.

**Data Availability:** All scenario topologies, generator configurations, result artifacts, seeds, and execution logs supporting the findings of this study are available in the replication package archived at [REPOSITORY-URL], persistent identifier `[TODO(DOI): mint a Zenodo DOI before submission]`. The two entry points that regenerate this paper's tables are:

```
PYTHONPATH=. python reproduce/detection_validation.py --layer app \
    --output results/detection_validation_app.json
PYTHONPATH=. python reproduce/run_prescribe_all.py --kappa 1.0
```

The first regenerates Tables 5 and 6; the second regenerates Tables 8 and 9. The prescriptive sweep takes approximately 2 h 47 min on 20 cores.

**Author Contributions:** [To be completed via the submission interface, per the journal's Submitting Declarations requirements.]

**Ethics Approval:** Not applicable.

# References


Aldeida Aleti, Barbora Buhnova, Lars Grunske, Anne Koziolek, and Indika Meedeniya. Software architecture optimization methods: A systematic literature review. *IEEE Transactions on Software Engineering*, 39(5):658–683, 2013.

Alberto Avritzer, Andrea Janes, Catia Trubiani, Helena Rodrigues, Yuanfang Cai, Daniel Sadoc Menasché, and Álvaro José Abreu de Oliveira. Architecture and performance anti-patterns correlation in microservice architectures. In *Proc. IEEE 22nd International Conference on Software Architecture (ICSA)*, pages 60–71, 2025. doi: 10.1109/ICSA65012.2025.00016.

Umberto Azadi, Francesca Arcelli Fontana, and Davide Taibi. Architectural smells detected by tools: A catalogue proposal. In *Proc. IEEE/ACM International Conference on Technical Debt (TechDebt)*, pages 88–97, 2019. doi: 10.1109/TechDebt.2019.00027.

Alexey Bakhtin, Marcelo Esposito, Valentina Lenarduzzi, and Davide Taibi. Network centrality as a new perspective on microservice architecture. In *Proc. IEEE International Conference on Software Architecture (ICSA)*, pages 72–83, 2025.

Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016. doi: 10.1109/MS.2016.60.

Steffen Becker, Heiko Koziolek, and Ralf Reussner. The Palladio component model for model-driven performance prediction. *Journal of Systems and Software*, 82(1):3–22, 2009. doi: 10.1016/j.jss.2008.03.066.

William H. Brown, Raphael C. Malveau, Hays W. McCormick, and Thomas J. Mowbray. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. John Wiley & Sons, 1998.

Yuanfang Cai and Rick Kazman. Dv8: Automated architecture analysis tool suites. In *Proc. IEEE/ACM International Conference on Technical Debt (TechDebt)*, pages 53–54, 2019. doi: 10.1109/TechDebt.2019.00015.

Benoit Combemale, Julien DeAntoni, Benoit Baudry, Robert B. France, Jean-Marc Jézéquel, and Jeff Gray. *Globalizing Domain-Specific Languages*, volume 9400 of *Lecture Notes in Computer Science*. Springer, 2016.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2): 182–197, 2002.

Daniel H. M. Falci, Omar A. Gomes, and Fernando S. Parreiras. Complex networks analysis for software architecture: An hibernate call graph study. *IEEE Access*, 6:62145–62155, 2018.

Peter H. Feiler, David P. Gluch, and John J. Hudak. The architecture analysis & design language (AADL): An introduction. Technical Report CMU/SEI-2006-TN-011, Software Engineering Institute, Carnegie Mellon University, 2006.

Francesca Arcelli Fontana, Ilaria Pigazzini, Riccardo Roveda, Damian Tamburri, Marco Zanoni, and Elisabetta Di Nitto. Arcan: A tool for architectural smells detection. In *Proc. IEEE International Conference on Software Architecture Workshops (ICSAW)*, pages 282–285, 2017. doi: 10.1109/ICSAW.2017.16.

Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.

Linton C. Freeman. A set of measures of centrality based on betweenness. *Sociometry*, 40(1): 35–41, 1977.

Mark Harman, S. Afshin Mansouri, and Yuanyuan Zhang. Search-based software engineering: Trends, techniques and applications. *ACM Computing Surveys*, 45(1):1–61, 2012.

ISO/IEC. Iso/iec 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model. Technical report, International Organization for Standardization, 2023a.

ISO/IEC. Iso/iec 25019:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model. Technical report, International Organization for Standardization, 2023b.

Anne Koziolek, Heiko Koziolek, and Ralf Reussner. Peropteryx: Automated application of tactics in multi-objective software architecture optimization. In *Proc. Joint ACM SIGSOFT Conference on Quality of Software Architectures and ACM SIGSOFT Symposium on Architecting Critical Systems (QoSA-ISARCS)*, pages 33–42, 2011. doi: 10.1145/2000259.2000267.

Sanghoon Lee, Junsoo Kang, and Kyung-Joon Park. Dependency chain analysis of ros 2 dds qos policies: From lifecycle tutorial to static verification. *IEEE Internet of Things Journal*, 12(4): 3890–3901, 2025a.

Sanghoon Lee, Hyun-Seok Park, Jiseok Chae, and Kyung-Joon Park. Probabilistic latency analysis of the data distribution service in ros 2. *IEEE Transactions on Industrial Informatics*, 21(3):2415–2426, 2025b.

Valentina Lenarduzzi, Francesco Lomio, Heikki Huttunen, and Davide Taibi. Are SonarQube rules inducing bugs? In *Proc. IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 501–511, 2020. doi: 10.1109/SANER48275.2020.9054821.

Jiaxin Li, Yuxi Chen, Haopeng Liu, Shan Lu, Yiming Zhang, Haryadi S. Gunawi, Xiaohui Gu, Xicheng Lu, and Dongsheng Li. PCatch: Automatically detecting performance cascading bugs in cloud systems. In *Proc. 13th European Conference on Computer Systems (EuroSys)*, pages 1–14, 2018. doi: 10.1145/3190508.3190552.

Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall, 2003.

Tom Mens and Pieter Van Gorp. A taxonomy of model transformation. *Electronic Notes in Theoretical Computer Science*, 152:125–142, 2006. doi: 10.1016/j.entcs.2005.10.029.

Tim Menzies and Foyzur Rahman. Bad smells in software analytics papers. *IEEE Software*, 35 (4):100–103, 2018.

Ran Mo, Yuanfang Cai, Rick Kazman, and Lu Xiao. Hotspot patterns: The formal definition and automatic detection of architecture smells. In *Proc. 12th Working IEEE/IFIP Conference on Software Architecture (WICSA)*, pages 51–60, 2015. doi: 10.1109/WICSA.2015.12.

Haris Mumtaz, Paramvir Singh, and Kelly Blincoe. A systematic mapping study on architectural smells detection. *Journal of Systems and Software*, 173:110885, 2021. doi: 10.1016/j.jss.2020.110885.

Shrideep Pallickara, Hasan Bulut, and Geoffrey Fox. Fault-tolerant reliable delivery of messages in distributed publish/subscribe systems. In *Proc. 4th IEEE International Conference on Autonomic Computing (ICAC)*, pages 12–21, 2007.

Riccardo Pinciroli, Aldeida Aleti, and Catia Trubiani. Performance modeling and analysis of design patterns for microservice systems. In *Proc. IEEE 20th International Conference on Software Architecture (ICSA)*, pages 35–46, 2023. doi: 10.1109/ICSA56044.2023.00012.

Douglas C. Schmidt. Model-driven engineering. *IEEE Computer*, 39(2):25–31, 2006. doi: 10.1109/MC.2006.58.

Tushar Sharma, Pratibha Mishra, and Rohit Tiwari. Designite: A software design quality assessment tool. In *Proc. 1st International Workshop on Bringing Architectural Design Thinking into Developers' Daily Activities (BRIDGE)*, pages 1–4, 2016. doi: 10.1145/2896935.2896938.

Connie U. Smith and Lloyd G. Williams. Software performance antipatterns. In *Proc. 2nd International Workshop on Software and Performance (WOSP)*, pages 127–136, 2000. doi: 10.1145/350391.350420.

Connie U. Smith and Lloyd G. Williams. New software performance AntiPatterns: More ways to shoot yourself in the foot. In *Proc. International Computer Measurement Group Conference (CMG)*, pages 667–674, 2002.

Cesar A. Stuardo, Tanakorn Leesatapornwongsa, Riza O. Suminto, Huan Ke, Jeffrey F. Lukman, Wei-Chiu Chuang, Shan Lu, and Haryadi S. Gunawi. ScaleCheck: A single-machine approach for discovering scalability bugs in large distributed systems. In *Proc. 17th USENIX Conference on File and Storage Technologies (FAST)*, pages 359–373, 2019.

Girish Suryanarayana, Ganesh Samarthyam, and Tushar Sharma. *Refactoring for Software Design Smells: Managing Technical Debt*. Morgan Kaufmann, 2014.

Davide Taibi, Valentina Lenarduzzi, and Claus Pahl. Microservices anti-patterns: A taxonomy. In *Microservices: Science and Engineering*, pages 211–228. Springer, 2020.

Catia Trubiani, Riccardo Pinciroli, Andrea Biaggi, and Francesca Arcelli Fontana. Automated detection of software performance antipatterns in Java-based applications. *IEEE Transactions on Software Engineering*, 49(4):2873–2891, 2023. doi: 10.1109/TSE.2023.3234321.

Guozhang Wang, Joel Koshy, Sriram Subramanian, Kartik Paramasivam, Mammad Zadeh, Neha Narkhede, Jun Rao, Jay Kreps, and Joe Stein. Building a replicated logging system with apache kafka. *Proceedings of the VLDB Endowment*, 8(12):1654–1655, 2015.

Ibrahim Onuralp Yigit and Feza Buzluca. A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems. In *Proc. IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE)*, 2025.

Hao-Nan Zhu, Goodness Ayinmode, Cesar A. Stuardo, Haryadi S. Gunawi, and Cindy Rubio-González. Understanding and detecting scalability faults in large-scale distributed systems. *IEEE Transactions on Software Engineering*, 52(10):3166–3179, 2026. doi: 10.1109/TSE.2026.3710096.